Université Mohammed V de Rabat

École Nationale Supérieure d'Arts et Métiers

Département Mathématiques Appliquées & Génie Informatique



Rapport de projet

Conception et développement d'un système de génération intelligente de devis multi-modal

Texte, photo et vocal pour l'estimation de travaux du bâtiment

Réalisé par : [**Nom et prénom**]

Dans le cadre de la formation : [**Filière / parcours**]

Effectué au sein de : [**Entreprise / organisme d'accueil**]

Période : [**Date de début**] **-** [**Date de fin**]

Encadré par : [**Nom de l'encadrant académique**] et [**Nom de l'encadrant professionnel**]

Année universitaire : **2026/2027**

Remerciements

Je tiens à remercier l'ensemble des personnes ayant contribué à la réalisation de ce projet, notamment mes encadrants académique et professionnel, ainsi que les collaborateurs ayant partagé leur expertise métier dans le domaine du chiffrage des travaux. Leurs retours ont permis d'ancrer la solution dans les besoins réels de préparation et de validation des devis.

Résumé

Ce rapport présente la conception et le développement de *Devis IA*, un système de génération intelligente de devis pour les travaux du bâtiment. La solution transforme des demandes formulées en texte, à l'oral ou à partir d'une photographie en prestations de devis structurées. Elle s'appuie sur un catalogue PostgreSQL d'entreprise, sur des modèles de langage pour l'identification des prestations et de leurs options, ainsi que sur des mécanismes déterministes de calcul de quantités, de coûts et de prix de vente.

Le projet couvre l'interface de saisie, une API FastAPI, une base PostgreSQL et un frontal React. La modalité photo associe l'analyse visuelle à une mesure géométrique fondée sur un rectangle de référence noir de dimensions connues. La modalité vocale s'appuie sur un modèle ASR français libre et hébergeable, sélectionné après consultation de benchmarks publics selon le critère du plus faible taux d'erreur de mots (WER). Pour la modalité texte, une optimisation d'instructions avec GEPA a été menée à l'aide d'un évaluateur métier inspiré d'ExtractBench, qui évalue séparément l'identification des prestations, les quantités, les options et les choix.

Mots-clés : devis, bâtiment, intelligence artificielle, LLM, GEPA, PostgreSQL, traitement d'image, reconnaissance vocale, multimodalité.

Abstract

This report presents the design and development of *Devis IA*, an intelligent quotation generation system for construction work. The system converts text, voice and photo-based requests into structured quotation services. It relies on a company PostgreSQL catalogue, language models for service and option extraction, and deterministic rules for quantities, costs and selling prices.

Keywords : quotation, construction, artificial intelligence, LLM, GEPA, PostgreSQL, image processing, speech recognition, multimodality.

Glossaire

Terme	Définition
API	Interface de programmation permettant la communication entre le frontal et les services métier.
ASR	*Automatic Speech Recognition* : transcription automatique d'un signal vocal en texte.
F1	Mesure combinant précision et rappel.
GEPA	Méthode d'optimisation d'instructions pilotée par l'évaluation des sorties.
GT	*Ground Truth* : réponse de référence attendue.
LLM	Grand modèle de langage.
RAG	Recherche de passages pertinents dans une base afin d'enrichir un prompt.
WER	*Word Error Rate* : taux d'erreur de mots d'un système de transcription.

Table des matières

Table des matières iii

Liste des tableaux iv

1 Introduction 1

2 Présentation du projet 1
2.1 Contexte métier . . . 1
2.2 Objectif général . . . 1
2.3 Périmètre et limites . . . 1

3 Cahier des charges 2
3.1 Spécifications fonctionnelles . . . 2
3.2 Spécifications de données . . . 2
3.3 Contraintes non fonctionnelles . . . 2

4 Organisation et méthodologie 2
4.1 Démarche de réalisation . . . 2
4.2 Outils de travail . . . 2
4.3 Jeux de données d'évaluation . . . 2

5 Conception de l'architecture 3
5.1 Architecture globale . . . 3
5.2 Schéma de sortie du modèle de langage . . . 3
5.3 Chiffrage . . . 3
5.4 Mesure depuis une photographie . . . 4
5.5 Modalité vocale . . . 4
5.6 Recherche sémantique et embeddings . . . 4

6 Réalisation technique et implémentation 4
6.1 Pipeline texte . . . 4
6.2 Interface de devis . . . 4
6.3 Gestion des erreurs . . . 5
6.4 Assurance qualité . . . 5

7 Résultats et discussion 5
7.1 Évaluateur métier pour l'optimisation des prompts . . . 5
7.2 Résultats disponibles de l'optimisation GEPA . . . 5
7.3 Emplacement réservé aux résultats consolidés de l'optimisation . . . 6
7.4 Protocole du benchmark final . . . 6
7.5 Résultats de test fonctionnel . . . 7
7.6 Discussion . . . 7

8 Conclusion générale et perspectives 7

Liste des tableaux

2 Responsabilités des principaux composants 3
3 Emplacement réservé au benchmark des modèles d'embedding 4
4 Synthèse des observations disponibles dans les résultats GEPA 6
5 Résultats détaillés GEPA à insérer 6
6 Emplacement réservé au benchmark final des modèles retenus 6
7 Emplacement réservé aux cas de test applicatifs 7

1. Introduction

La préparation d'un devis de travaux demande de comprendre une demande parfois imprécise, de sélectionner les opérations pertinentes, de renseigner les variantes techniques et d'établir un prix cohérent. Cette activité combine vocabulaire métier, contraintes de catalogue et calculs de coûts. Lorsqu'elle est réalisée manuellement, elle mobilise du temps et peut produire des écarts entre le besoin exprimé par le client et les lignes retenues dans le devis.

Le projet *Devis IA* répond à cette problématique par un système multi-modal de génération assistée de devis. Le commercial peut décrire les travaux au clavier, dicter sa demande ou fournir une photographie. Le système produit ensuite des occurrences de prestations reliées au catalogue réel de l'entreprise, puis prépare un devis contrôlable par un utilisateur métier.

L'enjeu n'est pas seulement de produire un texte plausible. La sortie de l'intelligence artificielle doit respecter une structure JSON stricte, utiliser des identifiants réels de la base de données et préserver les relations entre prestations, options et choix. Le reste de ce rapport décrit le besoin, l'architecture, les modalités développées, le protocole d'évaluation et les résultats actuellement disponibles.

2. Présentation du projet

2.1. Contexte métier

L'entreprise utilise un catalogue de prestations de travaux, de matériaux, de main-d'oeuvre et d'options techniques. Les demandes reçues peuvent être détaillées, partielles ou formulées oralement. Le projet vise à accélérer la création d'un brouillon de devis sans remplacer la validation du professionnel.

2.2. Objectif général

L'objectif est de concevoir une application capable de transformer une description de chantier en lignes de devis exploitables. La solution doit :

— reconnaître une ou plusieurs prestations dans une demande ;

— extraire une quantité lorsqu'elle est explicitement donnée ou directement calculable ;

— sélectionner uniquement les options et choix présents dans le catalogue ;

— calculer les coûts et les prix de vente à partir des composants disponibles ;

— présenter un aperçu clair, produire un PDF et préparer l'envoi au client ;

— proposer des entrées texte, photo et vocales.

2.3. Périmètre et limites

Le système assiste la préparation du devis. Les montants, les prestations proposées et les quantités restent vérifiables et modifiables par le commercial. Une quantité absente de la demande n'est pas assimilée à zéro ; elle est conservée à `null` jusqu'à validation ou saisie complémentaire.

3. Cahier des charges

3.1. Spécifications fonctionnelles

Le système doit permettre de créer un brouillon de devis depuis une description de travaux, consulter et ajuster les prestations détectées, choisir les options proposées par le catalogue et calculer le montant selon les composants associés. L'aperçu destiné au client doit afficher les composants sélectionnés et non une duplication artificielle d'un prix unitaire de prestation lorsqu'il n'existe pas dans la base.

Une prestation sans composition est traitée explicitement : l'interface permet la saisie d'un coût puis le prix HT est calculé selon la quantité et la marge retenue. Cette règle évite de présenter un prix fictif lorsque le catalogue ne fournit pas de composants matériels ou de main-d'oeuvre.

3.2. Spécifications de données

Le catalogue PostgreSQL constitue la source de vérité. Il contient 97 prestations, 141 options et 584 choix. Les relations *prestation* $\rightarrow$ *option* $\rightarrow$ *choix* sont contrôlées lors de l'exploitation de la sortie du modèle. L'identifiant seul ne suffit donc pas : une option doit appartenir à la prestation sélectionnée et un choix à cette option.

3.3. Contraintes non fonctionnelles

Les sorties générées doivent être syntaxiquement valides, traçables et robustes face aux erreurs d'API. Les clés d'accès aux fournisseurs de modèles ne sont pas intégrées au code source. Les temps de réponse, les erreurs et les coûts estimés sont journalisés afin d'observer le comportement de chaque modalité.

4. Organisation et méthodologie

4.1. Démarche de réalisation

Le développement a été mené de manière itérative : mise en place de la base et du pipeline métier, remplacement du frontal, intégration des modalités photo et vocale, puis conception des protocoles d'évaluation. Chaque évolution a été vérifiée avec des cas de chantier représentatifs et des contrôles de cohérence du catalogue.

4.2. Outils de travail

Le projet s'appuie sur Python et FastAPI pour l'API, React pour l'interface, PostgreSQL pour les données métier et OpenCV pour le module de mesure d'image. Les expérimentations d'optimisation et de benchmark des LLM sont exécutées dans Kaggle avec des jeux de données JSON versionnés.

4.3. Jeux de données d'évaluation

Un dataset synthétique de 60 descriptions a été construit à partir du catalogue réel : 30 exemples servent à l'optimisation GEPA et 30 à la validation. Les 12 descriptions originales destinées au test final sont séparées de ce dataset, afin d'éviter de mesurer la performance sur des formulations déjà vues pendant l'optimisation.

5. Conception de l'architecture

5.1. Architecture globale

L'architecture sépare l'interface utilisateur, l'API et les services de données. Le frontal React recueille la demande et affiche le brouillon. L'API FastAPI orchestre l'appel au modèle, valide le JSON retourné, vérifie les identifiants dans PostgreSQL et transmet les occurrences au moteur de chiffrage. La base conserve le catalogue et les devis.

Table 2 – Responsabilités des principaux composants

Composant	**Responsabilité**
Interface React	Saisie texte, chargement photo, enregistrement vocal, édition et aperçu du devis.
API FastAPI	Orchestration, validation, gestion des erreurs, génération et envoi du devis.
Catalogue PostgreSQL	Prestations, options, choix, composants, coûts et relations de référence.
Service LLM	Extraction des prestations, quantités, options et choix dans un JSON strict.
Moteur de chiffrage	Calcul des quantités de composants, coût total, prix de vente HT et marges.
Module photo	Calibrage par rectangle de référence, mesure d'une zone sélectionnée et analyse visuelle.
Module vocal	Transcription du signal audio avant passage dans le pipeline textuel.

5.2. Schéma de sortie du modèle de langage

Le modèle ne renvoie pas directement un devis libre. Il doit produire un objet JSON dont la racine contient uniquement la liste des prestations. Chaque prestation contient son identifiant, une quantité ou `null`, ainsi que les couples option/choix sélectionnés.

```
{
  "prestations": [
    {
      "prestation_id": 1,
      "quantite_ouvrage": 8,
      "options": [{"option_id": 1, "choix_id": 2}]
    }
  ]
}
```

5.3. Chiffrage

Pour une occurrence ayant une quantité q, le moteur calcule pour chaque composant sa quantité à partir de sa consommation par unité d'ouvrage. Le coût total est la somme des coûts des composants. Le prix HT est obtenu à partir du coût et de la marge commerciale, avant application de la TVA. Lorsque la prestation est dépourvue de composition, un coût saisi dans l'interface est nécessaire pour déclencher le même mécanisme de calcul.

5.4. Mesure depuis une photographie

Le commercial saisit les dimensions réelles du rectangle noir de référence, puis place ce rectangle sur l'élément à mesurer dans la photographie. Le module détecte le repère, corrige la perspective et permet à l'utilisateur de sélectionner les limites de l'élément. Les points sélectionnés sont reliés visuellement dans l'interface afin de rendre la zone mesurée explicite. Cette mesure fournit une estimation d'aire ou de longueur qui doit être validée sur chantier.

5.5. Modalité vocale

La piste vocale applique une chaîne ASR avant le pipeline de génération de devis. Des benchmarks publics de modèles gratuits, auto-hébergeables et adaptés au français ont été consultés. Le modèle Hojo a été retenu dans cette phase de sélection car il présentait le WER le plus faible parmi les modèles considérés. Les transcriptions sont ensuite traitées comme les descriptions textuelles.

5.6. Recherche sémantique et embeddings

La solution prévoit une recherche sémantique dans le catalogue afin de limiter le contexte fourni au LLM. Le choix définitif du modèle d'embedding doit être justifié par une comparaison reproductible.

TABLE 3 – Emplacement réservé au benchmark des modèles d'embedding

Modèle	**Métrique de recherche**	**Latence**	**Décision**
à compléter	*à compléter*	*à compléter*	*à compléter*
à compléter	*à compléter*	*à compléter*	*à compléter*
à compléter	*à compléter*	*à compléter*	*à compléter*

6. Réalisation technique et implémentation

6.1. Pipeline texte

La description est envoyée avec un prompt d'instructions et le catalogue pertinent. Le backend analyse ensuite la réponse selon trois étapes : validation du JSON, vérification des références dans le catalogue et construction d'un brouillon de devis. Une réponse formellement valide mais contenant un identifiant absent ou une relation incohérente est signalée, plutôt que transformée silencieusement en ligne de devis.

6.2. Interface de devis

Le frontal présente les occurrences de prestations, leurs composants et leurs calculs. Les fourchettes affichées sont rapportées à la quantité d'ouvrage sélectionnée et non artificiellement à une unité fixe. L'aperçu PDF et le document envoyé au client suivent la même présentation afin d'éviter un écart entre la validation à l'écran et le résultat transmis.

6.3. Gestion des erreurs

Les appels externes peuvent échouer, notamment pour cause de quota ou de limite de débit. Ces erreurs sont distinguées des réponses produites par le modèle. Elles sont enregistrées avec le fournisseur, le modèle, la latence et le détail de l'erreur ; aucune métrique métier n'est calculée pour une requête qui n'a pas reçu de réponse API exploitable.

6.4. Assurance qualité

Les tests du pipeline contrôlent la transformation d'une demande en brouillon, le calcul des composants et la cohérence des identifiants. Les tests de benchmark distinguent la qualité d'une réponse obtenue, la conformité JSON, la latence et la disponibilité de l'API.

7. Résultats et discussion

7.1. Évaluateur métier pour l'optimisation des prompts

L'optimisation GEPA repose sur un évaluateur personnalisé inspiré de la méthode ExtractBench. Les prestations répétées sont appariées globalement avec l'algorithme hongrois de `scipy.optimize.linear_sum_assignment` ; le même principe est appliqué aux options de prestations appariées. L'ordre d'apparition dans le JSON n'influence donc pas le score.

Trois F1 indépendants sont calculés : `prestation_id_f1`, `quantite_ouvrage_f1` et `options_choix_f1`. La conformité JSON est suivie séparément. L'évaluateur conserve également les extraits de preuve du dataset ainsi que les valeurs attendues, manquantes, incorrectes et supplémentaires pour alimenter le diagnostic du modèle de réflexion.

La frontière GEPA est configurée avec `frontier_type="cartesian"`. La règle d'admission impose de ne pas régresser sur la conformité JSON ni sur l'identification des prestations. La sélection finale exige une conformité JSON de 100%, maximise ensuite le F1 des prestations puis privilégie les instructions les plus courtes. Seuls les tokens d'instructions sont comptés, à l'exclusion des descriptions et du catalogue variable.

7.2. Résultats disponibles de l'optimisation GEPA

Les quatre modèles de tâche ont utilisé le même modèle de réflexion : *NVIDIA Nemotron 3 Ultra 550B*. Les deux candidats NVIDIA suivants ont été écartés avant le benchmark final, car leur conformité JSON restait inférieure à 100% dès l'évaluation initiale : *nvidia/nemotron-3.5-lightning-30b-a3b* et *nvidia/nemotron-3.5-bolt-e4b*. Ils ne sont donc pas comparés dans l'évaluation finale des modèles retenus.

TABLE 4 – Synthèse des observations disponibles dans les résultats GEPA

Modèle de tâche	Conformité JSON observée	Décision	Observation
nvidia/nemotron-3.5-lightning-30b-a3b	maximum observé : 96,67%	Écarté	Cinq candidats finaux recensés ; aucun n'atteint 100% de conformité JSON.
gemini-3.1-flash-lite	100% pour le candidat initial	Retenu	Candidat initial : F1 prestations 100%, F1 quantité 86,67%, F1 options/choix 100%, 198 tokens d'instructions.
gemini-3.5-flash-lite	100% sur les sous-échantillons observés	À confirmer au benchmark final	Aucune nouvelle mutation visible ; l'initialisation obtenait déjà un score maximal sur les sous-échantillons évalués.
nvidia/nemotron-3.5-bolt-e4b	maximum observé : 96,67%	Écarté	La conformité JSON est restée inférieure à 100% dans le pool observé.

7.3. Emplacement réservé aux résultats consolidés de l'optimisation

Le tableau suivant est volontairement laissé à compléter avec les scores complets exportés pour les quatre modèles. Il permet de conserver dans le rapport les résultats exacts de chaque exécution.

TABLE 5 – Résultats détaillés GEPA à insérer

Modèle	JSON	F1 prest.	F1 quant.	F1 opt./choix	Tokens instr.
gemini-3.1-flash-lite	–	–	–	–	–
gemini-3.5-flash-lite	–	–	–	–	–
nvidia/nemotron-3.5-lightning-30b-a3b	–	–	–	–	–
nvidia/nemotron-3.5-bolt-e4b	–	–	–	–	–

7.4. Protocole du benchmark final

Le benchmark final compare les modèles retenus sur les 12 descriptions originales, avec dix appels indépendants par description. Les métriques de qualité ne sont calculées que lorsque l'API a effectivement renvoyé une réponse. La conformité JSON est également évaluée uniquement sur les réponses obtenues.

La performance et la disponibilité sont dissociées : la latence est évaluée par la médiane des appels réussis de chaque description, avec un seuil de 30 secondes, tandis que la disponibilité correspond au pourcentage d'appels API réussis sur les 120 tentatives par modèle. La capacité théorique est calculée à partir des quotas RPM, TPM et RPD réels. Le score global combine qualité métier (40%), conformité JSON (20%), latence (10%), disponibilité (10%) et capacité théorique (20%).

TABLE 6 – Emplacement réservé au benchmark final des modèles retenus

Modèle	Score métier	JSON	Latence	Disponibilité	Score global
–	–	–	–	–	–
–	–	–	–	–	–

7.5. Résultats de test fonctionnel

Cette section est réservée aux captures, cas d'essai et résultats de validation fonctionnelle de l'application : génération textuelle, transcription vocale, mesure photo, édition de devis, aperçu PDF et envoi email.

TABLE 7 – Emplacement réservé aux cas de test applicatifs

Fonctionnalité	**Cas testé**	**Résultat attendu**	**Résultat obtenu**
à compléter	*à compléter*	*à compléter*	*à compléter*
à compléter	*à compléter*	*à compléter*	*à compléter*
à compléter	*à compléter*	*à compléter*	*à compléter*

7.6. Discussion

Le projet met en évidence l'importance de l'évaluation structurée pour les LLM appliqués à un catalogue métier. Une réponse naturelle peut sembler correcte tout en étant inexploitable si elle ne respecte pas le schéma ou si elle référence une option appartenant à une autre prestation. La séparation entre qualité, conformité, latence et disponibilité permet de ne pas confondre une erreur métier avec une indisponibilité temporaire du fournisseur de modèle.

8. Conclusion générale et perspectives

Le système *Devis IA* propose une chaîne cohérente allant de la demande client au brouillon de devis : compréhension multi-modale, sélection contrôlée des éléments du catalogue, calcul de prix et restitution dans une interface métier. L'usage d'identifiants PostgreSQL réels et la validation des relations entre prestations, options et choix renforcent la fiabilité de la génération.

L'optimisation des instructions avec GEPA fournit un cadre reproductible pour améliorer le module texte. Le choix des modèles ne repose pas sur une impression qualitative : il tient compte de la conformité JSON, de l'identification des prestations, de la quantité, des options, de la latence et de la disponibilité. Les emplacements réservés dans ce rapport permettront d'ajouter les résultats consolidés du benchmark des embeddings, du benchmark final des LLM et des tests applicatifs.

Les perspectives principales sont l'intégration du modèle d'embedding retenu, l'enrichissement du corpus de test réel, la validation terrain des mesures photographiques et la mise en place d'un suivi longitudinal des performances des modèles et des coûts d'inférence.

Références à compléter

1. Documentation GEPA et protocole d'optimisation utilisé dans les expérimentations.
2. Publication ou dépôt de référence ExtractBench concernant l'appariement hongrois des structures répétées.
3. Source des benchmarks publics utilisés pour le choix du modèle ASR français Hojo.
4. Documentation des fournisseurs de modèles et quotas utilisés dans le benchmark final.